Milestone 2

1. (Alexandra) Problem and data: What task will your system handle, which dataset will you use, and how will it support ingesting new data?

Problem: The system takes a spread as input, such as a 3-card or Celtic Cross, along with the drawn cards, their orientation (upright or reversed), and any optional user context. It then generates a single, coherent reading where each card is interpreted based on its position and how it relates to the others. Users can ask follow-up questions that build on that reading.

Data: We'll use all 78 cards, including upright and reversed meanings, keywords, and suit or arcana. Our main source is Tarotoo dataset based on the Pictorial Key to the Tarot (Waite, 1910, public domain). For each spread, we'll create a JSON file with its name, number of cards, the meaning of each position (like Past, Present, or Future), and a theme (general, love, career, mood or spiritual). We'll also write short guide texts about card combinations, suits, and reversals to help with interpretations beyond single-card meanings, including about 10-15 combination texts for well-known card pairs.

Ingesting new data: Adding new decks, spreads, or articles will be done by dropping files into data/, then running one ingestion script (parse, chunk, embed, upsert into the vector DB). Nothing needs to be rebuilt by hand, as each chunk will carry metadata (source, card, suit, element, arcana, cards for combination texts, orientation, type).

2. (Rebe) Golden Dataset: How will you create a set of representative questions and expected answers for evaluation?

Size: at most 100 examples, split into a set for tuning and one for testing, for safekeeping

Types: single-card lookups, reversed cards, position-specific meaning, full multi-card readings, personalized readings (with user context), and follow-up questions

Source: card meanings come from the Tarotoo open dataset (based on Waite's Pictorial Key to the Tarot >> https://www.kaggle.com/datasets/tarotoo/tarotoo-tarot-card-meanings). We then split the 78 cards three ways, normalize each batch, and trim it to 3/4 clear keywords per orientation.

How to DIY:

(~60) have a script randomly sample a spread, cards and orientation then lookup each card's keywords and fill a template ([card] [orientation] [position] : {keywords}. The script also runs the same analyze_spread code that the agent uses (dominant suit or elements, number of reversed cards, notable pairs) and stores its outputs as connection points. About a third of the generated examples deliberately include a pair that has a combination text, so that notable-pair detection is tested.

(~40) will be manual labor, we manually write follow-up questions and personalized readings since a script can't do this part well. Each follow-up example stores a fixed 'previous reading' as part of its input.

Expected answers will be stored as a list of keypoints that must appear in the reading, since an exact match would be useless for freestyle text. If an example has user context, a keypoint (say ‘refer to user’s situation’) will be automatically added.

Each example has:

Card Points: what it means by itself in the given position and orientation

Connection Points: the relationships found by analyze_spread that the reading has to mention

Context Points: added automatically if the example has user context

Expected Behaviour: which tools the agent should call, which spread it should use, and which guide chunks retrieval should return

e.g. The Tower | Reversed | Past >> avoiding change | delaying inevitable || with a follow up question of the sort ‘Does that mean the divorce is on me?’

Metrics:

Retrieval hit rate: does the Vector DB return the correct card and orientation chunk? (as in do the retrieved chunks contain the expected ones)

Keypoint coverage: have an LLM judge receive the generated reading and the list of keypoints and answers y/n for each one, the score being easily computable. We report card points and connection points separately, since connection coverage is what shows whether the SLM treats cards as a connected reading instead of isolated entries.

Correct orientation/position usage: simple code check that every card is mentioned with the correct orientation and position

Faithfulness/Connection: (LLM judge) does the reading contradict the retrieved card’s meaning? | check if the reading relates at least 2 cards to each other.

Agent Decisions: code check that compares the logged tool calls and the chosen spread with the expected ones.

Mentioning the fact that the LLM judge(DeepSeek) will only be an evaluation step and will have no place in the final product.

3. (Alexandra) RAG design: What chunking strategy, embedding model, vector database, and RAG approaches will you use or compare? Explain why.

Chunking:

Structure-aware chunking is our primary strategy, with three kinds of chunks:

Card chunks: one chunk per card orientation (for example “The Tower – reversed”). Each holds the general meaning plus the love, career, mood and spiritual fields for that orientation. get_card_meaning returns the general meaning and the field that matches the theme of the card’s position.

Combination chunks: one chunk per combination text (about 10-15 well-known pairs). Each carries the two cards it covers in its metadata, which is how analyze_spread detects notable pairs.

Guide chunks: the suit and reversal guide texts, split section by section.

All chunks carry metadata (source, card, suit, element, arcana, cards for combination texts, orientation, type).As a baseline for comparison, we also build fixed-size chunks (around 300 tokens, 50 overlap).

Why? Because the cards are distinct entities, selecting one card for each chunk prevents mixing the meanings. This also addresses the issue we found, which is that SLMs treat cards as separate entries. To address this, we combine the prompt's per-card chunks. So one chunk per card orientation, then all card chunks are combined in the prompt so the SLM sees the cards together.

Embedding model: we compare all-MiniLM-L6-v2 (small and fast) with bge-small-en-v1.5 (usually better retrieval, around the same size class). They both run locally.

Vector DB: ChromaDB operates locally; it doesn't require a server and allows metadata filtering, which is just enough for our small-sized data.

RAG methods for comparison:

Naive: only semantic top-k

Metadata-filtered: semantic search, but just for the guidance texts, and precise lookup of each drawn card and orientation.

Hybrid: BM25 + dense retrieval, to see if keyword matching helps with card names

Since we already know which cards were selected and semantic search can only introduce errors there, our hypothesis is that the second one prevails.

4. (Deni) Agent and tools: What decisions will the agent make, what tools will it use, and how do you plan to expose those tools through an MCP server?

The agent is an orchestrator around the SLM. Code runs the parts of a reading that must be right every time, and the SLM makes the judgment calls and writes the text. Instead of one prompt-in, text-out call, the SLM makes these decisions:

Spread selection: if the user doesn't name a spread, it calls list_spreads and picks the one that fits best (e.g. a one-card spread for a quick question, three-card for past/present/future, Celtic Cross for a complex situation).

Personalization: it decides whether it has enough personal information to personalize the reading or whether to ask one clarifying question first. It also turns what the user shares into structured context and stores it with update_user_context.

Follow-up routing: for each follow-up, it decides whether to answer from the stored reading, look up a card, or search the guides.

Besides these decisions, the SLM writes the reading itself (one connected interpretation of all cards, using the retrieved meanings and connection notes) and the answers to follow-ups.

What is fixed: the main reading follows a fixed sequence of tool calls driven by code: validate, look up meanings, search guides, analyze, generate, save. We chose this because validation, lookups, counting patterns and saving state must be correct every time, and small models are unreliable at sequencing many tools. The SLM's tool-calling decisions are measured with the agent-decisions check in the golden dataset.

Tools:

list_spreads -> shows the available spreads with their descriptions and card counts

get_spread -> loads a spread's layout as an ordered list of positions, each with its meaning and theme

validate_reading -> checks that the submitted cards fit the chosen spread (right number of cards, no duplicates, orientation given for each)

get_card_meaning -> looks up the curated meaning of a card for its orientation: the general meaning plus the field matching the position's theme (love, career, mood or spiritual)

search_guides -> searches the guide texts (card combinations, suits, reversals) so the agent can retrieve knowledge beyond single-card meanings

analyze_spread -> detects patterns across the drawn cards using fixed rules: the dominant suit and element, the number of Major Arcana cards, the ratio of reversed cards, and notable pairs (pairs of drawn cards that have a combination text in the guides), so the SLM gets explicit connections to narrate

save_reading -> stores the full structured reading (cards, positions, orientations, user context, interpretation) so it can be reused later

load_reading -> restores a saved reading so follow-up questions keep the original context

update_user_context -> stores personal details the user shares so the reading can be personalized

Which tool serves which follow-up: a question that can be answered from the reading itself uses only load_reading. A question about one card's detail ("what does the reversed Tower mean for my job?") uses get_card_meaning. A question about how cards interact ("how do the Tower and the Star relate?") uses search_guides for the combination and suit texts.

MCP exposure: We plan to build a single tarot MCP server. Its core is the nine functions above, registered as MCP tools, each with a name, a description and a typed input schema. The agent calls them to act: validating a reading, looking up a card, searching the guides, analyzing the spread, saving or loading state. As optional extras, the server can also expose read-only resources through URIs (such as tarot://cards/{id}, tarot://spreads/{id} and tarot://readings/{id}) and reusable prompt templates (such as interpret_reading and follow_up) so the SLM always receives the cards, connection notes and user context in the same structure. We will implement the tools first and add resources and prompts only if time allows.

The SLM handles the language side: judgment calls, personalization, and writing. The card knowledge, the pattern analysis and the reading state all live in the server. This also means the SLM can be swapped for another model without changing any tool.

5. (Deni) Model choices: Which SLM or other models are you considering, and what led you to choose them?

Some variants we are considering are: Qwen 3 4B, Gemma 4 (e2b or e4b), Phi-4 Mini (3.8B). However, because we're already familiar with Qwen, we're thinking about settling for that. Even so, here are the reasons why we're considering all of these models:

Qwen3 4B: our team is already familiar with it, so we can start building quickly. It is designed with function calling and agentic use in mind, which our MCP-based agent depends on. The family also comes in several sizes (0.6B, 1.7B, 4B, 8B), so we can test how small the model can be without changing the rest of the pipeline.

Gemma 4 (E2B or E4B): it's ranked among the strongest small open-weight models at the 2B-4B scale, so it may produce the most natural and fluent readings. The E2B variant is also small enough to run on very limited hardware.

Phi-4 Mini (3.8B): It is reported to be strong at reasoning and structured output, which matters because our SLM has to follow connection notes and tool results in a fixed format.

Judge model: Separately from the SLMs above, we will use a larger model, DeepSeek (accessed through its API), as an LLM judge. It is used only for evaluation, and it is never part of the tarot system. We chose it because it is much larger than the SLMs we evaluate, so it is better at catching subtle errors, and because it belongs to a different family than our main SLM (Qwen), which reduces bias toward its own style.

6. (Rebe) Plan: Show a timeline week by week from this moment through the Week 14 progress final presentation, including the experiments you plan to complete.

ASAP: Baseline experiment: the same SLM with a plain prompt, no tools or RAG.

Warning: Risks and fallbacks list (tool calling unreliable, SLM output quality) might cause the schedule to slip.

Week 6: Submit this doc. Gather the card and spread data, and decide how it's stored.

Week 7: Build the script that loads data into the vector DB and try two ways of cutting the text up.

Week 8: Build the golden dataset and a script that checks if search finds the right cards.

Week 9: Build the MCP server. Compare the embedding models and chunking styles.

Week 10: Connect the first SLM so it can do a full reading using the tools. Compare the 3 RAG approaches.

Week 11: Add follow-up questions and personalization. Compare the 3 SLMs. Pick the best one obviously.

Week 12: Last sprint before the milestone in case something got delayed.

Week 13: Last experiments (e.g. system with no RAG, to prove RAG helps). Run the test set. Fix problems.

Week 14: Make results tables, clean up the README, prepare the presentation.

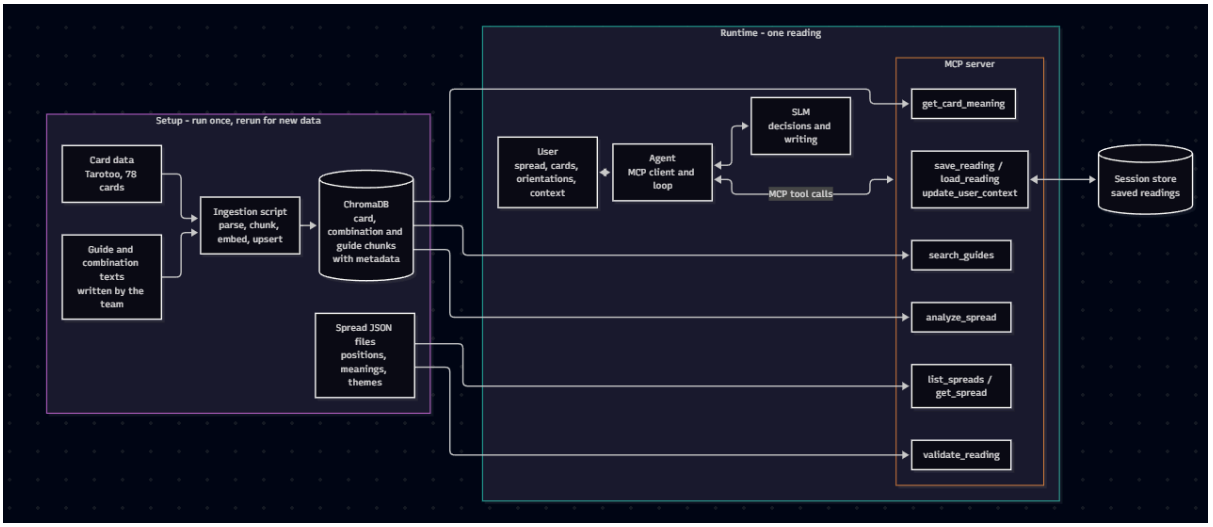
7. (Alexandra) Suggestion from git: diagram for one reading end to end: how the selected spread, card positions, and reversals become inputs, how the SLM connects the cards, and how follow-up questions retain the original reading context

Most of what this question asks was already covered in points 1 to 6, so we won't repeat it here: the spread JSON files and the per-card-orientation chunks (Data and RAG design), and the tools for loading spreads, retrieving cards, and saving the reading (Agent and tools). For one reading, end to end, this is how it would look:

1. Spread and context. The user picks a spread (for example a 3-card Past/Present/Future or a Celtic Cross) and can add optional personal context. If they don't pick one, the agent calls list_spreads and suggests the one that fits. The agent then calls get_spread, which returns the ordered positions with their meanings and themes, and update_user_context stores any personal details.

2. Cards in. The user enters the whole reading as a list: each card from their physical deck with its orientation and position (for example, “Present: The Tower, reversed”). The agent does not draw cards, it only interprets them.
3. Validation. validate_reading checks the full list against the 78 cards and the spread: the right number of cards, no duplicates, and an orientation for every card. If something is missing or invalid, the agent asks the user to fix it before continuing.
4. Looking up all the meanings (reversals). For every card in the list, get_card_meaning retrieves the chunk that matches the card, its orientation and the theme of its position. A reversed card retrieves its reversed chunk (“The Tower – reversed”), so the reversed meaning is already in the retrieved text and the SLM never has to flip an upright meaning itself.
5. Searching the guides. search_guides retrieves the guide texts that are relevant to this particular spread, such as combination texts for the card pairs that appear, suit texts, and texts about reversals. This gives the SLM knowledge beyond single-card meanings.
6. Connection analysis. analyze_spread looks at all the cards together, using plain code and no SLM. It produces explicit connection notes: the dominant suit and element, the number of Major Arcana cards, the ratio of reversed cards, and notable pairs (pairs of drawn cards that have a combination text in the guides).
7. One generation (connecting the cards). The agent assembles a single prompt containing the question, the user’s context, the position meanings, all the retrieved card meanings, the guide texts and the connection notes. The SLM writes one reading that goes through the positions in order and says how each card relates to the others, ending with a short summary. The relationships are computed by code in step 6, so the SLM only has to narrate them, not discover them.
8. Saving once. After the reading is generated, save_reading stores the full structured reading (spread, cards, positions, orientations, user context, connection notes, the interpretation and a short summary) in a session store.
9. Follow-up questions. Each follow-up calls load_reading to restore the stored reading, instead of relying on the chat history, which small models handle poorly. If the question can be answered from the reading itself, the agent answers directly. If it asks about one card’s detail, the agent calls get_card_meaning first. If it asks how cards interact, the agent calls search_guides first. The SLM then answers using the stored reading plus the newly retrieved text.

The diagrams:



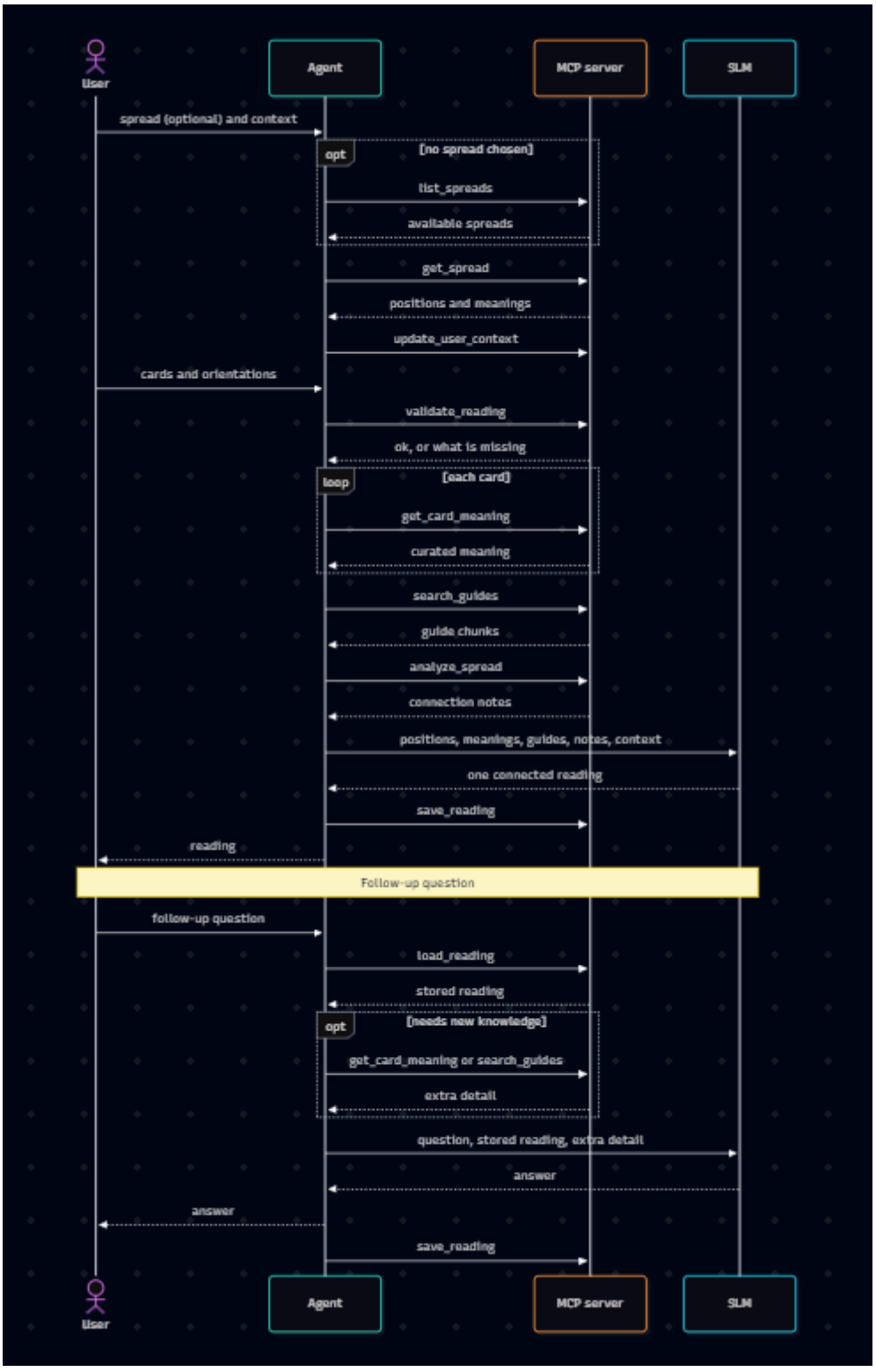
The code for the diagram:

```
flowchart LR
    subgraph SETUP["Setup - run once, rerun for new data"]
        D1["Card data<br/>Tarotoo, 78 cards"] --> ING["Ingestion script<br/>parse, chunk, embed, upsert"]
        D2["Guide and combination texts<br/>written by the team"] --> ING
        ING --> VDB[("ChromaDB<br/>card, combination and guide chunks<br/>with metadata")]
        D3["Spread JSON files<br/>positions, meanings, themes"]
    end

    subgraph RUNTIME["Runtime - one reading"]
        U["User<br/>spread, cards, orientations, context"] <--> AG["Agent<br/>MCP client and loop"]
        AG <--> SLM["SLM<br/>decisions and writing"]
        AG <-->|"MCP tool calls"| SRV

        subgraph SRV["MCP server"]
            T1["list_spreads / get_spread"]
            T2["validate_reading"]
            T3["get_card_meaning"]
            T4["search_guides"]
            T5["analyze_spread"]
            T6["save_reading / load_reading<br/>update_user_context"]
        end
    end

    D3 --> T1
    D3 --> T2
    VDB --> T3
    VDB --> T4
    VDB --> T5
    T6 <--> SS[("Session store<br/>saved readings")]
```



Code for sequence diagram bcs of quality:

```
sequenceDiagram
    actor U as User
    participant A as Agent
    participant M as MCP server
```

```
participant S as SLM

U->>A: spread (optional) and context
opt no spread chosen
    A->>M: list_spreads
    M-->>A: available spreads
end
A->>M: get_spread
M-->>A: positions and meanings
A->>M: update_user_context
U->>A: cards and orientations
A->>M: validate_reading
M-->>A: ok, or what is missing
loop each card
    A->>M: get_card_meaning
    M-->>A: curated meaning
end
A->>M: search_guides
M-->>A: guide chunks
A->>M: analyze_spread
M-->>A: connection notes
A->>S: positions, meanings, guides, notes, context
S-->>A: one connected reading
A->>M: save_reading
A-->>U: reading

Note over U,S: Follow-up question
U->>A: follow-up question
A->>M: load_reading
M-->>A: stored reading
opt needs new knowledge
    A->>M: get_card_meaning or search_guides
    M-->>A: extra detail
end
A->>S: question, stored reading, extra detail
S-->>A: answer
A-->>U: answer
A->>M: save_reading
```